PROJECT DOCUMENTATION

Job Aggregator Platform

Career Aggregation Platform for Students

Project Team

Haarish Ahmad M U	24011103015
Sai Chaithanyia P	24011103039
Aadithyaa E	24011103001
Anurag Subramaniyan	24011103005
Valan E	24011103066

GitHub Repository

github.com/Haarish-Ahmad/Job-Aggregator-Platform

October 31, 2026

Contents

1 Application Overview **1**

1.1 What the Application Does 1

1.1.1 Main Features 1

1.2 Frontend and Backend Feature Map 3

2 System Architecture and Workflow **4**

2.1 How the System Works 4

2.2 Container Architecture 4

2.3 Services and API 5

2.4 Technology Stack 6

3 Installation and Execution **7**

3.1 Requirements 7

3.2 Option A: Build from Source 7

3.3 Option B: Run Published Images from GHCR 8

3.4 How Images Are Published 9

3.5 Deploying to a Server 9

3.5.1 What the Setup Needs 9

3.5.2 Setting Up the Server 9

3.5.3 How a Deployment Works 10

3.5.4 What Is Stored on the Server 11

3.5.5 How the Scraper Reaches Firecrawl 11

3.5.6 Reaching the Site 11

4 Performance and Monitoring **12**

4.1 What Is Monitored 12

4.2 Measured Results 12

5 Testing and Validation **14**

5.1 Automated Tests 14

5.2 How It Was Tested 14

5.3 The CI Pipeline 15

6 Repository and Team Contributions **16**

6.1 Repository 16

6.2 Team Contributions 17

Conclusion **18**

Chapter 1

Application Overview

1.1 What the Application Does

The Job Aggregator Platform collects off-campus jobs and internships for students into one place. It scrapes job boards and company career pages, keeps only software roles a student can apply for, and matches each posting to the student's skills.

User	What they can do
Students	Register and sign in, maintain an academic/skills profile, upload a resume, browse jobs, review eligible matches, track applications, and opt into email alerts
Placement cell	Review hiring and matching analytics, skill demand and gaps

1.1.1 Main Features

- **Job Collection:** Jobs are collected from 8 registered source adapters:
  - Amazon Jobs
  - Google Careers
  - Microsoft Careers
  - Shine
  - Internshala
  - Unstop
  - Greenhouse
  - Y Combinator

  The scheduled scraper enables six by default: Amazon, Google, Microsoft, Shine, Internshala and Unstop. Greenhouse and Y Combinator are available as adapters but are not in the default schedule.

- **Relevance Filter:** Keeps software roles open to undergraduates, and drops senior, PhD-only and non-software roles.
- **Deduplication and Freshness:** Scraper postings are identified by source and external job ID, refreshed when seen, and deactivated when expired or unseen for the configured stale period (14 days by default).
- **Eligibility and Matching:** Matches exclude jobs that fail known deadline, minimum-CGPA, allowed-branch or graduation-year requirements. Eligible jobs are ranked with semantic similarity scores from the `all-MiniLM-L6-v2` sentence-transformer model; a score is not a literal percentage of required skills possessed.
- **Job Search:** Search by title/company and filter by job type, location, geography (India, remote, India + remote, or anywhere), skill, graduation year, source, closing status and sort order. Jobs also include application links and posting/freshness metadata when available.
- **Curated Company Groups:** Listings can be filtered by Big Tech, Indian startups, fintech, AI/deep tech, finance/quant, IT services, and core engineering. These are overlapping curated groups, not a ranked Top 100 list, and they filter listings rather than limiting what the scraper collects.
- **Student Tools:** View personalized match and skill-gap statistics; see jobs recommended for early applications; and track applications through saved, applied, assessment, interview, offer, rejected or withdrawn, with private notes.
- **Resume Parsing:** Upload, replace and download a text-based PDF resume (up to 10 MB). The parser reads up to five pages and extracts recognized skills, CGPA and graduation year to prefill the profile; the student reviews and saves profile changes.
- **Email Alerts:** Students can opt in to email for new eligible matches scoring at least 60%; the number of matches per message is configurable.
- **Placement-Cell Tools:** Platform hiring/match overview, skills-in-demand and trends, company hiring patterns, and skill-gap.
- **Monitoring:** Prometheus and Grafana dashboards for the API and the scraper.

1.2 Frontend and Backend Feature Map

Capability	Frontend	Backend support
Account and profile	Register, sign in/out, edit branch, CGPA, graduation year and skills	Session-based authentication; profile endpoints
Job listings	Search, pagination, type/ scope/location/skill/grad-year/company-group/source filters, sorting, hide expired	`GET /jobs` with filters, `X-Total-Count`, and a maximum page size of 100
Job detail and apply	View a listing and open its original application URL	Authenticated `GET /jobs/{job_id}`
Personalized matches	Minimum-score slider and optional inclusion of zero-score jobs	Eligibility checks and semantic-embedding match score at `GET /me/matches`
Resume	Upload/replace PDF, display extracted data, download saved PDF	`POST /me/resume` and `GET /me/resume`; text PDFs only
Student dashboard	Match count, early-application recommendations, popular skills and skill gaps	`GET /me/stats`
Application tracker	Create/update/remove records and filter by status	Owner-private `/me/applications` endpoints
Analytics	Skills and company analytics for signed-in users; skill-gap	`/analytics/skills`, `/analytics/companies`, `/analytics/skill-gap`
Admin	Platform metrics, recent/early/late job lists	Admin-only `/admin/analytics` and `POST /company`; authenticated company listing

The frontend requires an account for live job data and personalized pages.

The public home page uses sample preview listings for signed-out visitors.

Chapter 2

System Architecture and Workflow

2.1 How the System Works

The default schedule starts a scrape on startup and then runs Amazon Jobs, Google Careers, Microsoft Careers, Shine, Internshala and Unstop every 12 hours, with a default cap of 300 discovered jobs per source per cycle. Greenhouse and Y Combinator adapters are also registered and can be run on demand.

The platform processes job postings through the following stages:

1. **Scrape.** The scraper visits each configured source on its schedule. It respects `robots.txt` and waits between requests. Amazon job listings are retrieved from the public `amazon.jobs` search JSON endpoint; because its location parameter may return global results, the adapter also filters locations locally.
2. **Clean and filter.** Each posting is converted to plain text, then checked by the relevance filter.
3. **Find skills and matches.** The scraper normalizes source-provided job details and skills. Backend matching embeds the student's skills/branch and each job's title, required skills and description using Sentence Transformers.
4. **Send to the backend.** Records go in batches to `POST /internal/jobs`, protected by a shared token. The backend answers per record: inserted, updated, unchanged or rejected. One bad record never fails the batch.
5. **Keep it fresh.** After a cycle delivers jobs to the backend, expired listings and scraper-sourced postings unseen for the configured stale period (14 days by default) are deactivated.
6. **Alert.** Students who opted in receive one email listing their new matches.
7. **Serve.** Students use the website, and the placement cell reads the analytics.

2.2 Container Architecture

The container-level architecture consists of the following relationships:

- The browser reaches the platform through nginx, the single entry point.
- nginx routes `/` to the frontend (React), `/api/` to the backend (FastAPI), and `/grafana/` to Grafana.
- The scraper scrapes Amazon, Google, Microsoft, Shine, Internshala and Unstop, and uses on-demand adapters for Greenhouse and Y Combinator.
- The scraper uses Firecrawl, which runs as a separate stack, to render Shine pages.
- The scraper sends jobs to the backend through `POST /internal/jobs`.
- The backend stores data in PostgreSQL and sends alert emails to Mailpit (development inbox).
- Prometheus collects metrics from the `/metrics` endpoints of the backend and the scraper.
- Grafana uses Prometheus as its metrics source.

2.3 Services and API

Service	Role	Address
`nginx`	Single entry point	`http://localhost/`
`frontend`	The website	through nginx
`backend`	The API	`/api/`, docs at `/api/docs`
`db`	PostgreSQL database	internal
`scraper`	Scheduled scraping	internal
`prometheus`	Stores metrics	`http://localhost:9090` (this machine only)
`grafana`	Dashboards	`/grafana/`
`mailpit`	Catches emails in development	`http://localhost:8025` (this machine only)

The main API capabilities are:

- **Authentication/Profile:** `/auth/register`, `/auth/login`, `/auth/logout`, `/me`, and `/me/update`.
- **Jobs/Matches:** `/jobs`, `/jobs/filter` (curated company groups), `/jobs/{job_id}`, `/me/matches`, and `/me/stats`.
- **Resume:** `POST /me/resume` to upload and parse, and `GET /me/resume` to download.
- **Applications:** `GET`, `POST`, `PATCH` and `DELETE /me/applications...`; statuses are saved, applied, assessment, interview, offer, rejected and withdrawn.
- **Analytics:** Signed-in users can access skills and company analytics with geography scopes; skill-gap and curriculum analytics and `/admin/analytics` require admin access.

- **Scraper/Operations:** Token-protected `POST /internal/jobs`, `/internal/maintenance/deactivate-stale`, and `/internal/alerts/run` (legacy alias: `/internal/trigger_alerts`); `/health` and `/metrics` support health checks and monitoring.

The API uses signed session cookies for user authentication.

Scraper ingestion requires the configured shared token or an authenticated admin session.

2.4 Technology Stack

Part	Technology
Frontend	React 19, TypeScript 5.9, Vite 7, React Router 7, Vitest
Backend/API	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2, Alembic, Uvicorn
Matching	Sentence Transformers (`all-MiniLM-L6-v2`), PyTorch and NumPy embeddings/similarity
Resume parsing	`pypdf` text extraction and a rule/vocabulary-based skill, CGPA and graduation-year parser
Database	PostgreSQL 16
Scraper	Python 3.12, httpx, Beautiful Soup, Pydantic, Tenacity, Typer
Rendered-page service	Firecrawl, run separately and used by Shine; other adapters use source pages/APIs
Email	Python SMTP client; Mailpit by default for local development, configurable SMTP for delivery
Web entry point	nginx (unprivileged frontend and proxy containers)
Monitoring	Prometheus and Grafana
Containers	Docker and Docker Compose
CI/CD	GitHub Actions and GitHub Container Registry (GHCR)

The frontend includes dedicated pages for job listings and filters, job details, profile and resume upload, personalized dashboard, application tracking, and admin analytics/company management.

Chapter 3

Installation and Execution

3.1 Requirements

- Docker with Compose v2.24 or newer.
- **Firecrawl**, only for the Shine source. Self-host it from https://github.com/firecrawl/firecrawl so that it answers on port 3002. The other sources work without it.

3.2 Option A: Build from Source

Clone the repository, enter the project directory and create the environment file:

```
git clone https://github.com/Haarish-Ahmad/Job-Aggregator-Platform.git
cd Job-Aggregator-Platform
cp .env.example .env
```

Fill in the required values in `.env`. Each one can be generated with:

```
python3 -c "import secrets; print(secrets.token_hex(24))"
```

```
POSTGRES_PASSWORD=...     # letters and digits only
SECRET_KEY=...
INGEST_TOKEN=...
```

Optionally set `ADMIN_EMAIL` and `ADMIN_PASSWORD` to create or promote an administrator on backend startup. Set a strong, private password before exposing the app. The default local Mailpit service captures alert emails; configure SMTP settings in `.env` for actual delivery. Change the default Grafana credentials before exposing Grafana beyond the local machine.

The default `SERVE_SOURCES` setting starts Unstop, Shine, Internshala, Google, Microsoft and Amazon on startup and repeats every 12 hours. The default `SERVE_LIMIT` is 300 discovered jobs per source per cycle. Change `SERVE_SOURCES` to a comma-separated list to select different registered adapters; `SERVE_LIMIT` changes the per-source cap. Greenhouse

and Y Combinator adapters are available for one-off runs even though they are not in the default schedule.

Then start everything and wait until every service reports healthy:

```
docker compose up -d --build
docker compose ps
```

Open	What
`http://<server_ip>`	The website. Jobs appear as each source completes its first run; the full initial cycle can take longer, especially for Shine
`http://<server_ip>/grafana/`	Dashboards (login `admin` / `admin` by default)
`http://<server_ip>:8025/`	Mailpit inbox for alert emails

To stop the stack:

```
docker compose down
```

Adding `-v` also deletes the data.

3.3 Option B: Run Published Images from GHCR

The project publishes its images to GitHub Container Registry (GHCR). Docker Hub is used only for the official base images the stack is built on, such as PostgreSQL, Prometheus, Grafana and Mailpit.

Image	Name
Backend	`ghcr.io/haarish-ahmad/job-aggregator-platform-backend`
Scraper	`ghcr.io/haarish-ahmad/job-aggregator-platform-scraper`
Frontend	`ghcr.io/haarish-ahmad/job-aggregator-platform-frontend`
nginx	`ghcr.io/haarish-ahmad/job-aggregator-platform-nginx`

You need the repository's `docker-compose.yml`, `docker-compose.prod.yml`, the `monitoring/` folder and a `.env` file.

Pull the images and start the production Compose configuration:

```
docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml pull
docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up -d
```

`IMAGE_TAG` in `.env` selects the version: `latest`, a commit such as `sha-1a2b3c4`, or a release such as `1.2.3`.

3.4 How Images Are Published

Publishing is automatic. On a push to `main` or a version tag such as `v1.2.3`, the Release workflow builds the four images and uploads them to GHCR using GitHub's built-in token.

Trigger	Image tags
Push to `main`	`latest` and `sha-<commit>`
Version tag `v1.2.3`	`1.2.3` and `sha-<commit>`

3.5 Deploying to a Server

The `server-setup/` folder turns a Linux machine into the server with one command, `bash setup.sh`. After that, pushes to `main` publish the images. Automatic deployment runs through the machine's GitHub Actions runner only when the runner is registered and the repository variable `DEPLOY_ENABLED=true`; otherwise the deploy job is skipped. See `server-setup/README.md` for details.

3.5.1 What the Setup Needs

- A Linux machine running Ubuntu or Debian on a 64-bit Intel or AMD processor. An old laptop works.
- A user account with `sudo` rights, and an internet connection.
- 8 GB of RAM or more, because Firecrawl alone uses several GB.
- About 15 GB of free disk space for the images, data and the Firecrawl build.
- A GitHub account with admin rights on the repository.
- The `main` branch must already contain the workflows, and Release must have published the images.

The `server-setup/` folder holds three files:

File	Run by	Purpose
`setup.sh`	You, once	Prepares the machine and connects it to GitHub
`deploy.sh`	GitHub Actions, on every deployment	Starts Firecrawl and the application
`README.md`	You	Step-by-step instructions

3.5.2 Setting Up the Server

Copy the `server-setup` folder to the machine, then run it as a normal user, not with `sudo`:

```
cd server-setup
bash setup.sh
```

The script performs the following steps:

1. Installs Git and Docker, and gives the user access to Docker.
2. Stops the machine from going to sleep, including when a laptop lid is closed.
3. Creates `~/job-aggregator/deploy.env` with new random passwords. This file stays on the server and is never stored in Git. An existing file is kept.
4. Installs the GitHub Actions runner as a service, so it starts by itself after a reboot. It asks for a one-time token from the GitHub page it shows.

The runner connects out to GitHub, so no router ports need to be opened.

Three steps then finish the setup on GitHub:

1. Add the repository variable `DEPLOY_ENABLED` with the value `true` (Settings → Secrets and variables → Actions → Variables).
2. Choose **Require approval for all external contributors** (Settings → Actions → General). The repository is public and the runner executes code on the server, so a pull request from a stranger must never run there without approval.
3. Start the first deployment by pushing to `main`, or by clicking **Run workflow** on the Release workflow.

3.5.3 How a Deployment Works

1. A change is pushed or merged to `main`.
2. Release builds the four application images on GitHub's computers and uploads them to GHCR, tagged with the commit.
3. GitHub gives the `deploy` job to the runner on the server. The job only runs for `main`, and only when `DEPLOY_ENABLED` is `true`.
4. `deploy.sh` clones Firecrawl at a pinned version if it is missing, and builds it only when that version changes. The first build can take a long time.
5. `deploy.sh` pulls the images for that commit, starts the stack, and waits for every service to report healthy. If a service does not become healthy, the deployment fails.
6. Prometheus and Grafana restart to pick up any monitoring changes, and older application images are removed to save disk space.

The server must be on and connected to the internet for a deployment to run. Data in the database and monitoring history is kept between deployments.

3.5.4 What Is Stored on the Server

Everything is under `~/job-aggregator`:

Path	Contents
`deploy.env`	Settings and passwords, readable only by the user
`app/`	Compose files and monitoring settings of the running version
`actions-runner/`	The GitHub Actions runner
`firecrawl/`	Firecrawl source code, built on the server

3.5.5 How the Scraper Reaches Firecrawl

Firecrawl runs as its own Docker stack beside the application. Its API has no login, so `deploy.sh` publishes it only on Docker's internal bridge address. The scraper reaches it at `host.docker.internal:3002`, and other computers on the network cannot. `deploy.sh` also sets Firecrawl's services to restart after a reboot, because Firecrawl does not do this by default.

3.5.6 Reaching the Site

- **Website:** The server's address on the local network.
- **Grafana:** `/grafana/`
- **Grafana User:** `admin`
- **Grafana Password:** Stored in `deploy.env`.

Making the site public needs an extra step, such as Cloudflare Tunnel. In that case, set `PUBLIC_URL` to the public address, and set `SESSION_COOKIE_SECURE=true` when the site uses HTTPS.

Chapter 4

Performance and Monitoring

4.1 What Is Monitored

Prometheus collects metrics from the backend and scraper every 15 seconds and keeps 15 days of history.

Grafana shows them on two dashboards:

- **App Health:** API status, requests per second, server error rate, response times, active jobs, students, records received and alert emails.
- **Scraper:** Status, jobs saved and failed, records sent to and refused by the API, pages fetched, run time per source, and time since the last success.

4.2 Measured Results

The figures below are a historical baseline from before Amazon, Google and Microsoft were added to the default schedule. They are retained for context and do not describe the current six-source configuration. Counts, timings, memory and image sizes change with the deployment and should be re-measured for a current report.

Measured on the development machine (7.6 GB of RAM) with the earlier stack running.

API, over a 2.8-hour window

Metric	Value
Requests	538
Server errors (5xx)	0
Response time	about 5 ms
Response time, p95	about 10 ms
Response time, p99	about 290 ms

Traffic in this window was light and mostly health checks, so these numbers show normal idle behaviour rather than capacity.

Scraper, average time for one source

Source	Time
Internshala	about 23 seconds
Unstop	about 48 seconds
Shine	about 15 minutes (each page is rendered by Firecrawl)

- **Data Collected:** This historical snapshot held 1,507 active postings: Internshala 812, Shine 387, Unstop 307, Microsoft 1. Amazon and Google were not yet scheduled in this snapshot.
- **Memory Used by the Stack:** About 710 MB in total, with Grafana the largest at about 257 MB. Firecrawl runs separately and is not included. It needs several GB more, so a server should have about 8 GB of RAM or more.
- **Docker Image Sizes:** Current local image sizes observed on October 31, 2026 are backend 9.57 GB, scraper 221 MB, frontend 74.7 MB, and nginx 73.7 MB. These are machine/build-specific; the older 299 MB backend figure is not representative of the current image. The backend image includes the Sentence Transformers/PyTorch model stack.

Chapter 5

Testing and Validation

5.1 Automated Tests

The test suites cover the following areas. Counts vary as tests are added; use the latest GitHub Actions run for current CI totals.

Test suite	Tool	Count	What it checks
Backend	pytest with PostgreSQL	See latest CI run	API routes, ingestion, alerts, analytics, resume parsing and migrations
Scraper	pytest with saved pages and fixtures	See latest CI run	Source adapters (including Amazon, Google and Microsoft), robots rules, filtering, skill search, push, scheduling and storage
Integration	pytest with PostgreSQL	See latest CI run	Scraper-to-backend ingestion with results read back through the API
Frontend	Vitest	See latest CI run	Pages and components, including listing filters, resume upload, applications, dashboards and admin tools
Smoke test	`scripts/smoke.sh`	See latest CI run	The whole running stack

5.2 How It Was Tested

- **Static Checks:** Ruff (Python lint), ESLint and TypeScript (frontend), and ShellCheck (shell scripts) run in CI. Backend and scraper type checking is not currently a CI step.
- **Fixtures Instead of Live Sites:** Scraper tests use saved pages and structured fixtures where applicable. They are fast and repeatable, and do not send traffic to

job sites.

- **Recent Source Validation (October 31, 2026):** The focused Amazon, Google, Microsoft and source-configuration tests passed (22 tests). A live Amazon API request was verified; a sample Microsoft internship scrape inserted two eligible jobs. A Google India internship query returned no results at the time of the check. Amazon postings requiring two or more years of experience were correctly excluded by the existing student-relevance filter in the sample run.
- **Real Database:** Backend and integration tests use a real PostgreSQL. Migrations are applied to an empty database, and `alembic check` confirms they match the code's models.
- **Whole-Stack Smoke Test:** The script starts a separate copy of the stack with a mock Firecrawl, then checks that:
  - every container is healthy,
  - nginx routing and security headers work,
  - private endpoints are blocked,
  - sign-in works,
  - a scrape run twice creates no duplicates,
  - an alert email arrives,
  - Prometheus and Grafana work.
- **Configuration Checks:** CI confirms that every setting is documented in `.env.example`, that the compose files are valid and refuse to start without secrets, and that no Windows line endings are present.
- **Deployment Scripts:** `deploy.sh` was tested through a temporary local registry. Data survived a new version, old images were removed, and a version that does not exist failed while the site stayed up. `setup.sh` was tested on a clean Ubuntu container.

5.3 The CI Pipeline

Every pull request and non-`main` branch push runs these jobs in order. A later stage starts only if the earlier ones pass; pushes to `main` are handled by the Release workflow.

1. Backend, Scraper, Integration, Frontend and Config checks run in parallel.
2. After those pass, all four application Docker images are built.
3. The smoke test runs on the whole stack.

The Release workflow publishes images on pushes to `main` and version tags. Its deployment job runs only for `main` when the self-hosted runner and `DEPLOY_ENABLED` repository variable are configured.

Chapter 6

Repository and Team Contributions

6.1 Repository

The project source code is hosted at:

https://github.com/Haarish-Ahmad/Job-Aggregator-Platform

The repository is organised as follows:

```
backend/        FastAPI application, backend configuration and dependencies
  main.py, models.py, schemas.py, db.py, matching.py, resume.py,
  alert.py, analytics.py, config.py, metrics.py, create_admin.py
  alembic/      Database migrations
  tests/        Backend unit tests
  Dockerfile, docker-entrypoint.sh, requirements*.txt, alembic.ini
  target_companies.py, curriculum.yaml
tests/integration/  Scraper-to-backend integration tests
scraper/        Eight source adapters, scheduling pipeline, fixtures and tests
frontend/       React app, pages and tests
infra/nginx/    Web entry-point and reverse-proxy configuration
monitoring/     Prometheus settings and Grafana dashboards
docs/           UI screenshots
scripts/        Smoke test and configuration checks
server-setup/   Linux server setup and deployment scripts
.github/        CI and Release workflows
docker-compose*.yml, .env.example
                Root-level stack orchestration and shared configuration
```

The checked-out root is the active application stack; the root `docker-compose.yml` launches its services and builds the API from `backend/`. Similarly named nested directories (`Job-Aggregator-Platform-infra_cicd/` and `Job-Aggregator-Platform-integration-full-stack/`) and their ZIP bundles are not additional services in the active Compose stack.

`target_companies.py` contains seven curated, potentially overlapping company groups. There is no single ranked or fixed Top 100 companies list in the current implementation.

6.2 Team Contributions

Member	Contribution
Haarish Ahmad M U – 24011103015	CI/CD pipeline and server setup
Sai Chaithanyia P – 24011103039	Backend
Aadithyaa E – 24011103001	Frontend
Anurag Subramaniyan – 24011103005	Scraper
Valan E – 24011103066	Infrastructure

GitHub profiles of the team members:

- Haarish Ahmad M U: https://github.com/Haarish-Ahmad
- Sai Chaithanyia P: https://github.com/chay123-crypto
- Aadithyaa E: https://github.com/AadithyaaE
- Anurag Subramaniyan: https://github.com/anuragg-subramaniyan-07
- Valan E: https://github.com/ValanEz

Conclusion

The Job Aggregator Platform gives students one place to find off-campus software jobs and internships.

It has eight registered source adapters, with six enabled in the default schedule, filters postings for relevance, matches them to each student's skills, and sends email alerts. Students can upload resumes, track applications and use personalized dashboards; the placement cell gets analytics on skills in demand and curriculum alignment.

The project includes Docker-based deployment, automated image publishing, a server setup script, monitoring dashboards, and tests across the backend, scraper, frontend, integration and whole-stack workflows.

CI runs on pull requests and non-`main` branch pushes; release workflows publish on `main` and version tags, with deployment conditional on server-runner configuration.